

## **Documentação Técnica – Backend GPPRH**

## Conteúdo

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| <b>Documentação Técnica – Backend GPPRH</b>                               | 1  |
| <b>1. Visão Geral do Projeto</b>                                          | 3  |
| 2. Estrutura de Diretórios e Responsabilidades                            | 3  |
| 2.1. Principais Arquivos                                                  | 6  |
| 3. Configurações e Ambiente                                               | 6  |
| 4. Autenticação e Autorização                                             | 7  |
| 4.1. Autenticação                                                         | 7  |
| 4.2. Autorização                                                          | 8  |
| 5. Bancos de Dados e Integrações                                          | 8  |
| 5.1. Banco Local (MySQL)                                                  | 8  |
| 5.2. Protheus (SQL Server)                                                | 8  |
| 5.3. Oracle                                                               | 8  |
| 5.4. Active Directory                                                     | 8  |
| 5.5. Google OAuth                                                         | 8  |
| <b>6. WebSocket e Eventos em Tempo Real</b>                               | 8  |
| 7. Infraestrutura (PM2 e Apache)                                          | 9  |
| 7.1. Gerenciamento de Processos com PM2                                   | 9  |
| 7.2. Proxy Reverso Apache com SSL                                         | 10 |
| 8. Segurança                                                              | 10 |
| 8.1. Medidas Atuais                                                       | 11 |
| 8.2. Melhorias Planejadas                                                 | 11 |
| 9. Execução e Logs                                                        | 11 |
| 9.1. Inicialização                                                        | 11 |
| 9.2. Logs                                                                 | 11 |
| 10. Apêndice: Exemplo de Arquivo <code>.env</code> (com valores omitidos) | 11 |
| Considerações Finais                                                      | 12 |

## 1. Visão Geral do Projeto

Este backend foi desenvolvido em **Node.js** utilizando **Express** como framework principal, seguindo os princípios de **Clean Architecture** e **Domain-Driven Design (DDD)**. O objetivo é fornecer APIs para os sistemas de gestão de vagas e processos seletivos (GPPRH), integrando-se com múltiplas fontes de dados e serviços externos.

### Tecnologias principais:

- Node.js (ambiente de execução)
- Express (web framework)
- JSON Web Token (JWT) para autenticação
- MySQL e SQL Server (Protheus) como bancos de dados
- LDAP (Active Directory) para autenticação de usuários corporativos
- Google OAuth como método alternativo de login
- WebSocket (em desenvolvimento) para eventos em tempo real
- PM2 para gerenciamento de processos
- Apache como proxy reverso com SSL

## 2. Estrutura de Diretórios e Responsabilidades

A estrutura de pastas segue uma organização modular e por camadas. A raiz do projeto é `src`.

`src/`

```
├── app.js          # Configuração do Express, middlewares e registro de rotas
├── server.js       # Ponto de entrada que inicializa o servidor HTTP
├── application/    # Módulo de aplicação (auth)
│   └── auth/
│       └── auth-session.service.js # Serviço de sessão (não utilizado? – pendente)
├── config/        # Configurações de conexão com serviços externos
│   ├── ad.js      # Configuração do Active Directory (LDAP)
│   ├── mysql.js   # Configuração do MySQL
│   ├── oracle.js  # Configuração do Oracle (não utilizado atualmente)
│   └── protheus.js # Configuração do SQL Server (Protheus)
├── errors/        # Classes de erros customizadas
│   ├── app.error.js
│   ├── bad-request.error.js
│   ├── unauthorized.error.js
│   └── validation.error.js
```

```
└─ infra/          # Infraestrutura técnica
|  └─ auth/
|     └─ jwt.service.js    # Geração e verificação de tokens JWT
|     └─ ldap-auth.service.js # Autenticação via LDAP (AD)
└─ middlewares/      # Middlewares globais e de proteção
|  └─ auth.middleware.js   # Validação do token JWT
|  └─ error.middleware.js   # Tratamento centralizado de erros
|  └─ permission.middleware.js # Verificação de permissões (canAll)
|  └─ upload.middleware.js  # Upload de arquivos (se aplicável)
└─ modules/          # Módulos de domínio (cada um com sua estrutura)
|  └─ global/         # Módulo global (funcionalidades comuns)
|     └─ controllers/  # Controllers do módulo
|         └─ auth.controller.js
|         └─ clpp.controller.js
|         └─ employee.controller.js
|         └─ domain/    # Entidades de domínio
|             └─ user.entity.js
|         └─ repositories/ # Camada de acesso a dados (vazio – talvez use serviços)
|             └─ routes.js    # Rotas do módulo global
|             └─ services/    # Lógica de negócio
|                 └─ auth.service.js
|                 └─ employee.service.js
|         └─ gpprh/        # Módulo principal de vagas e processos
|             └─ controllers/
|                 └─ auth.controller.js
|                 └─ gpprh.controller.js
|             └─ domain/
|                 └─ jobs/    # Subdomínio de jobs
|                     └─ job-status.enum.js
|                     └─ job-status.helper.js
|                     └─ job-status.meta.js
```

```
| | | | └─ job-status.transitions.js
| | | | └─ job.contract.js
| | | | └─ job.entity.js
| | | | └─ job.fields.js
| | | └─ user.entity.js
| | └─ providers/    # Provedores externos
| | └─ google-auth.service.js
| | └─ repositories/ # Acesso a dados
| | | └─ job.repository.js
| | | └─ user.repository.js
| | └─ routes.js
| | └─ services/
| | | └─ gpprh.service.js
| | | └─ job.service.js
| | └─ validators/
| | └─ job.validator.js
| └─ protheus/    # Módulo de integração com o Protheus (SQL Server)
| | └─ controllers/
| | | └─ protheus.controller.js
| | └─ repositories/
| | | └─ cost-center.repository.js
| | └─ routes.js
| | └─ services/
| | └─ protheus.service.js
| └─ consinco/    # Módulo planejado (não utilizado – será removido)
└─ utils/        # Funções utilitárias
| └─ date-converter.js
| └─ time-parser.js
| └─ validate-required-fields.js
└─ websocket/    # Módulo WebSocket (em desenvolvimento)
    └─ connectionManager.js
```

```

├── eventBus.js
├── eventQueue.js
├── events/
│   ├── chat.event.js
│   └── notification.event.js
├── routes/
│   └── event.routes.js  # Rota /ws/emit-event
└── websocketServer.js

```

## 2.1. Principais Arquivos

- **app.js**  
Configura o Express: middlewares globais (JSON, cookie-parser, CORS), registro de rotas dos módulos (/protheus, /gpprh, /global), tratamento de 404 e middleware de erro final. *Observação:* A CORS está configurada para permitir origens específicas e credenciais (cookies).
- **server.js**  
Lê o arquivo .env (na raiz do projeto), importa app, define a porta e inicia o servidor. Também testa a conexão com o Protheus (MSSQL) no startup.
- **infra/auth/jwt.service.js**  
Responsável por gerar tokens de acesso (duração curta) e refresh (duração longa), além de verificar e renovar tokens. Os tokens são assinados com segredos definidos no .env.
- **middlewares/**  
  - auth.middleware.js** extrai o token JWT (geralmente de um cookie httpOnly ou cabeçalho Authorization) e valida.
  - permission.middleware.js** contém a função canAll(roles) que verifica se o usuário autenticado possui as permissões necessárias (roles armazenadas no banco local).

## 3. Configurações e Ambiente

As variáveis de ambiente são carregadas via dotenv a partir de um arquivo .env localizado na raiz do projeto (acima de src). As principais variáveis utilizadas são:

PORT=4000

# Banco Protheus (MSSQL)

DB\_PROTHEUS\_HOST=...

DB\_PROTHEUS\_USER=...

DB\_PROTHEUS\_PASS=...

DB\_PROTHEUS\_DB=...

# MySQL

DB\_MYSQL\_HOST=...

DB\_MYSQL\_USER=...

DB\_MYSQL\_PASS=...

DB\_MYSQL\_DB=...

# Active Directory

AD\_URL=ldap://...

AD\_DOMAIN=...

AD\_BASEDN=...

# JWT

JWT\_ACCESS\_SECRET=...

JWT\_REFRESH\_SECRET=...

JWT\_EXPIRES\_ACCESS\_IN=30m

JWT\_EXPIRES\_REFRESH\_IN=3d

**Nota:** O arquivo `config/oracle.js` existe, mas o Oracle não está em uso atualmente. As configurações de MySQL e Protheus estão ativas.

## 4. Autenticação e Autorização

### 4.1. Autenticação

Dois métodos de autenticação estão disponíveis:

- **Active Directory (LDAP)**  
Realizado pelo serviço `infra/auth/ldap-auth.service.js`, que realiza um bind com as credenciais do usuário no AD configurado. A autenticação é apenas de login; não há verificação de grupos de segurança no AD.
- **Google OAuth**  
Implementado no provedor `modules/gpprh/providers/google-auth.service.js`. O fluxo OAuth retorna os dados do usuário, que são então mapeados para a entidade `user.entity` local. O cadastro de novos usuários ocorre na primeira autenticação.

Após a autenticação bem-sucedida, o sistema gera dois tokens JWT:

- **Access Token** – válido por 30 minutos (configurável via `JWT_EXPIRES_ACCESS_IN`).
- **Refresh Token** – válido por 3 dias (configurável via `JWT_EXPIRES_REFRESH_IN`).

Os tokens são enviados ao cliente como **cookies httpOnly** para proteção contra XSS. O refresh token permite a renovação do access token sem necessidade de novo login.

#### 4.2. Autorização

A autorização é baseada em **roles** (perfis) armazenadas no banco de dados local (MySQL). Os middlewares `authMiddleware` e `canAll` são aplicados nas rotas para verificar:

- **authMiddleware**: valida o token e anexa o usuário decodificado ao objeto `req.user`.
- **canAll(roles)**: verifica se o usuário possui pelo menos uma das roles especificadas.

Exemplo:

```
router.post('/job', authMiddleware, canAll(['JOB_CREATE']), GrppController.createJob);
```

As roles são gerenciadas via tabelas no banco de dados e podem ser atribuídas a usuários.

### 5. Bancos de Dados e Integrações

#### 5.1. Banco Local (MySQL)

Utilizado para armazenar dados de domínio: usuários, roles, permissões, etc. As conexões são configuradas em `config/mysql.js` e utilizadas pelos repositórios nos módulos.

#### 5.2. Protheus (SQL Server)

Acesso ao ERP Protheus via SQL Server. A configuração está em `config/protheus.js`, e os repositórios (como `cost-center.repository.js`) realizam consultas diretas. A conexão é testada no startup do `server.js`.

#### 5.3. Oracle

Embora exista o arquivo `config/oracle.js`, **não está em uso** atualmente. Pode ser considerado para futuras expansões.

#### 5.4. Active Directory

Apenas para autenticação via bind LDAP, conforme descrito.

#### 5.5. Google OAuth

Utilizado como provedor de identidade alternativo.

### 6. WebSocket e Eventos em Tempo Real

O módulo `websocket/` está **em desenvolvimento**, mas já possui uma estrutura básica:

- `websocketServer.js` – Cria um servidor WebSocket (porta 4001) e gerencia conexões.
- `connectionManager.js` – Controla os clientes conectados.
- `eventBus.js` – Um barramento de eventos interno que permite emitir eventos para todos os clientes ou para um subconjunto.
- `eventQueue.js` – Fila de eventos para processamento assíncrono.
- `routes/event.routes.js` – Expõe uma rota HTTP `/ws/emit-event` que permite que a API dispare eventos via `eventBus`:

```
router.post("/emit-event", (req, res) => {  
  const { event, payload, options } = req.body;  
  eventBus.emit(event, payload, options);  
  res.json({ ok: true });  
});
```

- **Eventos definidos:**
  - `user-logged-in` – disparado pelo serviço `notification.event.js` quando um usuário realiza login (via `axios` para a rota `/ws/emit-event`). O evento é então transmitido para outros clientes conectados, excluindo o próprio usuário (via opção `excludeUserId`).

Os clientes WebSocket se inscrevem automaticamente ao estabelecer a conexão. A autenticação no WebSocket também utiliza JWT (ainda em implementação).

## 7. Infraestrutura (PM2 e Apache)

### 7.1. Gerenciamento de Processos com PM2

O sistema é gerenciado pelo **PM2** com os seguintes processos (saída atual):

| id | name        | mode    | status | porta | descrição                        |
|----|-------------|---------|--------|-------|----------------------------------|
| 4  | api-gpprh   | cluster | online | 4000  | API principal (módulos)          |
| 6  | api-gpprh   | cluster | online | 4000  | segunda instância (cluster mode) |
| 3  | front-gpprh | fork    | online | 3000  | Frontend (Next.js ou similar)    |
| 5  | ws-gpprh    | fork    | online | 4001  | Servidor WebSocket               |

- **api-gpprh** em **cluster mode** permite escalonamento horizontal dentro de uma única máquina. Não há necessidade de sticky sessions porque as rotas HTTP são stateless (JWT armazenado em cookie, não em memória).
- **ws-gpprh** em **fork mode** é isolado; a comunicação entre processos não é compartilhada, mas o design atual não exige estado compartilhado entre as instâncias WebSocket.

## 7.2. Proxy Reverso Apache com SSL

O Apache atua como proxy reverso para todas as aplicações, utilizando SSL. A configuração do VirtualHost é:

```
<VirtualHost *:443>

    ServerName vagas.gpprh.com.br

    SSLEngine on

    SSLCertificateFile /etc/ssl/vagas.gpprh/vagas.gpprh.crt
    SSLCertificateKeyFile /etc/ssl/vagas.gpprh/vagas.gpprh.key
    SSLCertificateChainFile /etc/ssl/vagas.gpprh/vagas.gpprh.ca-bundle

    ProxyRequests Off

    ProxyPreserveHost On

    # Mapeamento das APIs

    ProxyPass /api/v1/protheus/ http://127.0.0.1:4000/protheus/
    ProxyPassReverse /api/v1/protheus/ http://127.0.0.1:4000/protheus/

    ProxyPass /api/v1/gpprh/ http://127.0.0.1:4000/gpprh/
    ProxyPassReverse /api/v1/gpprh/ http://127.0.0.1:4000/gpprh/

    ProxyPass /api/v1/global/ http://127.0.0.1:4000/global/
    ProxyPassReverse /api/v1/global/ http://127.0.0.1:4000/global/

    # WebSocket

    ProxyPass /ws ws://127.0.0.1:4001/
    ProxyPassReverse /ws ws://127.0.0.1:4001/

    # Frontend

    ProxyPass / http://127.0.0.1:3000/
    ProxyPassReverse / http://127.0.0.1:3000/

</VirtualHost>
```

- O prefixo `/api/v1` é usado externamente, mas internamente as rotas são relativas (`/protheus`, `/gpprh`, `/global`). O Apache faz a reescrita.
- O WebSocket é exposto via `/ws` e encaminhado para a porta 4001.

## 8. Segurança

### 8.1. Medidas Atuais

- **HTTPS:** O Apache fornece SSL/TLS, garantindo criptografia na comunicação.
- **JWT em cookies httpOnly:** Os tokens de acesso e refresh são armazenados em cookies com flag `httpOnly`, protegendo contra acesso via JavaScript.
- **CORS:** Configurado para permitir apenas origens específicas (localhost e domínio de produção) com credenciais.
- **Validação de entrada:** Realizada pelos validadores nos módulos (ex: `job.validator.js`).
- **Tratamento de erros:** Middleware centralizado que não expõe detalhes internos.

### 8.2. Melhorias Planejadas

- **CSRF (Cross-Site Request Forgery):** Atualmente não implementado. Recomenda-se adicionar tokens CSRF em formulários que modificam estado.
- **CSP (Content Security Policy):** Não configurado. Sugere-se definir políticas para mitigar ataques XSS.
- **Rate limiting:** Para evitar abusos, pode-se adicionar limitação de requisições por IP.
- **Monitoramento de logs e alertas:** Atualmente os logs são apenas os padrões do PM2. Seria benéfico integrar com ferramentas de monitoramento (ex: Sentry, Prometheus).

## 9. Execução e Logs

### 9.1. Inicialização

Os processos são iniciados via PM2. Comandos típicos:

```
pm2 start ecosystem.config.js    # se existir
pm2 start src/server.js --name api-gpprh -i 2    # cluster mode
pm2 start frontend/... --name front-gpprh
pm2 start websocket/websocketServer.js --name ws-gpprh
```

### 9.2. Logs

Os logs são gerenciados pelo PM2 e podem ser visualizados com:

```
bash
```

```
pm2 logs api-gpprh
```

Os arquivos de log residem em `~/pm2/logs/`. Não há ferramentas externas de agregação no momento.

## 10. Apêndice: Exemplo de Arquivo `.env` (com valores omitidos)

```
# Servidor
PORT=4000
```

*# Proteus (MSSQL)*

DB\_PROTHEUS\_HOST=

DB\_PROTHEUS\_USER=

DB\_PROTHEUS\_PASS=

DB\_PROTHEUS\_DB=

*# MySQL*

DB\_MYSQL\_HOST=

DB\_MYSQL\_USER=

DB\_MYSQL\_PASS=

DB\_MYSQL\_DB=

*# Active Directory*

AD\_URL=

AD\_DOMAIN=

AD\_BASEDN=

*# JWT*

JWT\_ACCESS\_SECRET=

JWT\_REFRESH\_SECRET=

JWT\_EXPIRES\_ACCESS\_IN=30m

JWT\_EXPIRES\_REFRESH\_IN=3d

## Considerações Finais

Esta documentação reflete a estrutura atual do backend, baseada nas informações fornecidas e nos esclarecimentos obtidos. O sistema está em produção, utilizando Node.js com arquitetura limpa, integrações com AD, Google OAuth, MySQL e SQL Server, e é gerenciado por PM2 com proxy Apache.

Qualquer atualização ou mudança futura deve ser refletida nesta documentação para manter o conhecimento alinhado.

---

**Gerado em:** 20/03/2026

**Autor:** Hygor Bueno